Parte 1.2

Orden temporal del algoritmo de Floyd

El algoritmo de Floyd calcula las distancias mínimas entre todos los pares de vértices.
Para cada vértice intermedio, comprueba si pasar por él permite encontrar un camino más económico.
Su orden temporal es (O(V^3)), porque utiliza tres ciclos que recorren los vértices: uno para el vértice intermedio,
otro para el origen y otro para el destino.
En este caso hay 4 ciudades, por lo que se realizan (4^3=64) combinaciones de índices en los ciclos principales.
En general, no se puede reducir el orden del algoritmo clásico de Floyd, aunque sí se pueden realizar mejoras prácticas,
como evitar cálculos innecesarios cuando no hay caminos posibles o utilizar algoritmos alternativos si solo se necesitan algunas distancias.
En este escenario, el costo es muy bajo porque hay pocas ciudades. Sin embargo, si la aerolínea incorporara muchas más ciudades,
el tiempo de ejecución crecería considerablemente, ya que el costo aumenta de forma cúbica.

Parte 2

1.

Matriz Inicial:

	M	C	B	P
M	0	∞	∞	150
C	300	0	∞	390
B	400	200	0	∞
P	∞	∞	410	0

Se aplica el algoritmo de Floyd, considerando cada ciudad como vértice intermedio.
Después de considerar Montevideo como intermediaria,
Buenos Aires puede llegar a Punta del Este pasando por Montevideo, con costo 400 + 150 = 550.

	M	C	B	P
M	0	∞	∞	150
C	300	0	∞	390
B	400	200	0	550
P	∞	∞	410	0

Después de considerar Colonia como intermediaria, la matriz no cambia, porque no se encuentran caminos más económicos.
Después de considerar Buenos Aires como intermediaria,
Punta del Este puede llegar a Montevideo con costo 410 + 400 = 810 y a Colonia con costo 410 + 200 = 610.

	M	C	B	P
M	0	∞	∞	150
C	300	0	∞	390
B	400	200	0	550
P	810	610	410	0

Finalmente, se considera Punta del Este como intermediaria. Montevideo puede llegar a Buenos Aires con costo 150 + 410 = 560 y a
Colonia con costo 150 + 410 + 200 = 760. Colonia puede llegar a Buenos Aires con costo 390 + 410 = 800.

Matriz final:

	M	C	B	P

```
M      0  760  560   150
C    300    0  800   390
B    400  200    0   550
P    810  610  410     0
```

Y los caminos minimos entre todas las parejas de ciudades es:

Origen: Montevideo

Destino	Costo mínimo	Trayecto
Montevideo	0	Montevideo
Colonia	760	Montevideo → Punta del Este → Buenos Aires → Colonia
Buenos Aires	560	Montevideo → Punta del Este → Buenos Aires
Punta del Este	150	Montevideo → Punta del Este

Origen: Colonia

Destino	Costo mínimo	Trayecto
Montevideo	300	Colonia → Montevideo
Colonia	0	Colonia
Buenos Aires	800	Colonia → Punta del Este → Buenos Aires
Punta del Este	390	Colonia → Punta del Este

Origen: Buenos Aires

Destino	Costo mínimo	Trayecto
Montevideo	400	Buenos Aires → Montevideo
Colonia	200	Buenos Aires → Colonia
Buenos Aires	0	Buenos Aires
Punta del Este	550	Buenos Aires → Montevideo → Punta del Este

Origen: Punta del Este

Destino	Costo mínimo	Trayecto
Montevideo	810	Punta del Este → Buenos Aires → Montevideo
Colonia	610	Punta del Este → Buenos Aires → Colonia
Buenos Aires	410	Punta del Este → Buenos Aires
Punta del Este	0	Punta del Este

2.

Para determinar la ubicación más conveniente, se suman las distancias mínimas desde las otras ciudades hasta cada posible ubicación del
taller, respetando el sentido de los vuelos.

Montevideo: 300 + 400 + 810 = 1510.
Colonia: 760 + 200 + 610 = 1570.
Buenos Aires: 560 + 800 + 410 = 1770.
Punta del Este: 150 + 390 + 550 = 1090.

Por tanto, la ciudad más conveniente para instalar el taller de mantenimiento es Punta del Este, ya que presenta el menor costo total de
traslado desde las otras tres ciudades, con 1090 unidades de distancia.

Parte 3

```
Lista<V> recuperarCamino(int origen, int destino) {
```

```
    Lista<V> camino ← lista vacía
    int actual ← destino

    si distancia[origen][destino] = ∞ entonces
        retornar camino
    fin si

    mientras actual ≠ origen hacer
        insertar actual al principio de camino
        actual ← predecesor[origen][actual]

        si actual = null entonces
            retornar lista vacía
        fin si
    fin mientras

    insertar origen al principio de camino
    retornar camino
}
```